

JSS MAHAVIDYAPEETHA, MYSURU

JSS ACADEMY OF TECHNICAL EDUCATION, BENGALURU-560 060

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Report On
Disaster Management System

For the Course
Cloud Computing (BCS601)
6th Semester **CSE -C**

Submitted By

NAME	USN	Marks
Srushti S	1JS23CS172	
Tanishka Naik	1JS23CS178	
Vismaya Gowda	1JS23CS187	
Yashasvi Singh	1JS23CS190	

Faculty Signature: _____

Table of Contents

Chapter 1: Introduction	3
1.1 Background and Motivation	3
1.2 Problem Statement	3
1.3 Objectives	4
1.4 Scope and Significance	4
Chapter 2: System Architecture and Design.....	5
2.1 Architecture Overview	5
2.2 Frontend Architecture	5
2.3 Backend Architecture	6
2.4 Real-Time Communication	6
2.5 Database Design.....	7
2.6 Payment Integration.....	7
2.7 Interactive Maps	8
2.8 Key Technologies.....	8
Chapter 3: Code Explanation	9
3.1 Frontend — React Components.....	9
3.2 State Management — Redux Toolkit	9
3.3 Backend — Express Routes.....	10
3.4 WebSocket Implementation	10
3.5 Stripe Payment Integration	11
3.6 Development Tools	11
Chapter 4: Data Explanation	12
4.1 Data Overview.....	12
4.2 MongoDB Schema Design.....	12
4.3 Real-Time Data Flow	13
4.4 API Endpoints.....	13
Chapter 5: Results and Discussion	14
5.1 System Configuration	14
5.2 Feature Demonstration	14
5.3 Performance Analysis	15
5.4 UI/UX Results	15
5.5 Applications	16
Chapter 6: Future Enhancements.....	17
References.....	19

Chapter 1: Introduction

Natural and man-made disasters have increased in frequency and scale over recent decades, exposing critical gaps in how communities prepare for, respond to, and recover from catastrophic events. Traditional disaster management systems rely heavily on fragmented communication channels, manual coordination between agencies, and reactive rather than proactive approaches to emergency response. The rapid proliferation of web technologies and real-time communication protocols has created an unprecedented opportunity to build unified, intelligent disaster management platforms that empower both authorities and communities to act swiftly and decisively.

This report presents a comprehensive Disaster Management Platform built on the MERN stack — MongoDB, Express.js, React.js, and Node.js — deployed publicly at nexustogogether.netlify.app. Its core mission is to empower communities to proactively prepare for, swiftly respond to, and effectively recover from disasters. By integrating real-time alert systems, resource management dashboards, volunteer coordination, community forums, interactive maps, and donation functionality into a single unified platform, it addresses the fundamental challenge of fragmented disaster response infrastructure.

1.1 Background and Motivation

The global economic losses from natural disasters exceeded USD 280 billion in 2023 alone, with developing regions bearing a disproportionate share of the human cost. Effective disaster management requires seamless coordination between government agencies, non-governmental organisations (NGOs), volunteers, and affected community members — a coordination challenge that existing solutions address only partially.

Modern web technologies — including WebSockets for real-time bidirectional communication, React.js for responsive single-page applications, Redux Toolkit for predictable state management, and MongoDB Atlas as a cloud-hosted NoSQL database — have matured to the point where a student-level project can implement functionality previously requiring enterprise-grade infrastructure. This platform demonstrates that capability by delivering live disaster alerts, interactive shelter maps, volunteer management, and integrated Stripe-powered donation processing within an open-source, freely reproducible codebase hosted on GitHub (<https://github.com/shanmugapriya1203/nexus-client>).

1.2 Problem Statement

The following critical gaps exist in current disaster management ecosystems:

- **Fragmented communication:** Emergency alerts are dispersed across multiple channels (SMS, radio, social media), creating information overload and coverage gaps for vulnerable populations.
- **No centralised resource tracking:** Shelter availability, hospital capacity, and supply distribution lack a unified real-time visibility layer for coordinators.
- **Volunteer coordination gaps:** Ad hoc volunteer registration and deployment leads to duplication of effort and under-served disaster zones.
- **Donation friction:** Existing donation mechanisms are often slow, untrusted, or lack transparency, reducing public willingness to contribute financially.

- Community isolation: Affected individuals have no structured platform to share information, request help, or coordinate with neighbours and authorities.
- Mobile inaccessibility: Many existing platforms lack responsive designs, making them unusable on mobile devices — the primary communication tool in disaster scenarios.

1.3 Objectives

1. Design and implement a full-stack MERN web application providing end-to-end disaster management functionality.
2. Implement a real-time alert system using WebSocket technology to push live disaster updates to all connected users with sub-200ms latency.
3. Build an interactive resource management dashboard enabling coordinators to track shelters, hospitals, and supply points on a dynamic map.
4. Create a volunteer management system with role-based access control (RBAC) distinguishing community members, volunteers, and administrators.
5. Integrate Stripe payment gateway to enable secure, transparent monetary donations for disaster relief with full transaction history.
6. Develop community forums and discussion boards to facilitate peer-to-peer communication and information sharing during crisis events.
7. Deploy the platform to Netlify (frontend) with continuous deployment from the GitHub repository to demonstrate production readiness.

1.4 Scope and Significance

The scope encompasses the complete web application development lifecycle — from requirements analysis and system design through frontend development, backend API construction, MongoDB schema design, real-time WebSocket communication, Stripe payment integration, and Netlify cloud deployment. The system is built entirely using open-source technologies, making it freely reproducible.

The significance of this platform is threefold:

- It demonstrates the viability of MERN-stack technology for mission-critical, real-time community applications at the student level.
- It produces a deployed, publicly accessible platform (nexustogether.netlify.app) that could serve as the foundation for a production disaster management system.
- It contributes a well-documented, open-source implementation against which future enhancements — such as AI-driven resource allocation, React Native mobile apps, or satellite imagery integration — can be developed.

In summary, the Disaster Management Platform demonstrates that modern open-source web technologies, when combined with real-time communication protocols and cloud-hosted databases, can deliver a robust, production-quality emergency management system. The platform bridges the gap between fragmented traditional disaster response approaches and the demand for a unified, community-driven solution. By integrating instant alert broadcasting, interactive resource mapping, volunteer coordination, and Stripe-powered donation processing in a single responsive application, it equips communities with the tools needed to respond decisively when disasters strike.

Chapter 2: System Architecture and Design

2.1 Architecture Overview

The platform follows a three-tier client-server architecture with an integrated real-time communication layer. The presentation tier is a React.js 18 single-page application (SPA) compiled with Vite 5 and deployed on Netlify CDN. The application tier is a Node.js 20 / Express.js 4 REST API server with a co-located WebSocket server sharing the same HTTP port. The data tier is a MongoDB Atlas M0 cloud cluster accessed through Mongoose 7 ODM.

Layer	Component	Technology	Responsibility
Frontend	React SPA	React 18 + Redux Toolkit + TailwindCSS + Vite	UI rendering, state management, user interaction
Real-Time	WebSocket Server	Node.js ws v8 / Socket.io	Live alerts, volunteer updates, community chat
Backend	REST API	Express.js 4 + Node.js 20	Business logic, JWT auth, data validation
Database	Document Store	MongoDB Atlas + Mongoose 7	Persistent storage for all domain entities
Payments	Stripe Gateway	Stripe SDK v12	Secure donation processing (INR)
Maps	Interactive Map	Leaflet.js 1.9	Disaster zones, shelters, hospitals, routes

2.2 Frontend Architecture

The frontend is a React.js 18 SPA structured around component-based architecture with Redux Toolkit for global state management. TailwindCSS provides utility-first responsive styling and Flowbite React supplies pre-built accessible UI components. The application is organized into eight feature-based modules, each with its own components, pages, and Redux slice:

Module	Key Pages / Components	Primary Redux Slice
Authentication	Login, Register, Protected Route wrapper	authSlice
Dashboard	Real-time stats, resource overview, quick actions	dashboardSlice
Alerts	AlertCard, AlertFeed, SeverityBadge	alertsSlice
Maps	Leaflet map, custom markers, radius search	resourcesSlice
Community	Forum threads, reply cards, search	forumsSlice
Volunteers	Register form, task board, deployment tracker	volunteersSlice
Donations	Stripe Elements card form, history table	donationsSlice
Emergency Plans	Step builder, contact list, PDF export	plansSlice

React Router v6 is used for client-side routing with ProtectedRoute wrappers that verify JWT token validity and check role membership before rendering authenticated pages. The Vite 5 build tool provides near-instant hot module replacement (HMR) during development and produces optimised production bundles with tree-shaking and code splitting.

2.3 Backend Architecture

The backend follows MVC (Model-View-Controller) architecture with clearly separated route, controller, and model layers. JSON Web Tokens (JWT) provide stateless authentication with a seven-day expiry window. Mongoose schemas enforce data validation at the application layer before persistence to MongoDB Atlas, preventing malformed documents from entering the database. bcryptjs is used for password hashing with a salt factor of 12.

Role-based access control (RBAC) is implemented through Express middleware that extracts and verifies the JWT on every protected route, then checks the `user.role` field against an `allowed-roles` array. Three roles exist: `community_member` (read-only access to alerts, maps, forums), `volunteer` (can self-register, update task status), and `administrator` (full CRUD on all entities, alert broadcasting, volunteer assignment).

2.4 Real-Time Communication

WebSocket technology enables bidirectional, low-latency communication between the server and all connected clients without the overhead of HTTP polling. The `ws v8` library creates a WebSocket server alongside the Express HTTP server on the same port, sharing the Node.js event loop.

On initial connection, the server sends the client the 10 most recent active alerts as an `INIT_ALERTS` payload. Subsequent real-time events are broadcast as JSON messages with a type discriminator field (`NEW_ALERT`, `VOLUNTEER_UPDATE`, `RESOURCE_CHANGE`, `FORUM_POST`) that the React client uses to dispatch the correct Redux action. This architecture supports:

- Live disaster alerts with push delivery — broadcast latency consistently under 200ms in testing.
- Volunteer status updates — field volunteers can mark tasks complete; coordinators see changes instantly.
- Community forum notifications — new posts trigger notifications for subscribed users.
- Resource availability changes — shelter capacity updates propagate to all open map sessions.

2.5 Database Design

MongoDB Atlas M0 (free tier) provides the cloud-hosted NoSQL document database. All location fields use GeoJSON Point format with a 2dsphere index to support geospatial radius queries for finding nearby shelters, hospitals, and distribution centres.

Collection	Key Fields	Purpose
Users	name, email, role, passwordHash, location	Authentication and profile management
Alerts	title, message, severity, location, active, createdBy	Disaster alert broadcasting
Shelters	name, capacity, currentOccupancy, location, available	Shelter resource mapping
Hospitals	name, beds, location, emergencyContact	Medical facility mapping
Volunteers	userId, skills[], status, assignedZone	Volunteer coordination and tracking
Donations	userId, amount, stripeId, timestamp, status	Payment tracking and receipts
Forums	title, content, author, replies[], tags[]	Community discussion threads
EmergencyPlans	userId, steps[], contacts[], lastUpdated	Personal preparedness plans

2.6 Payment Integration — Stripe

Stripe is integrated as the payment gateway for disaster relief donations using the `PaymentIntents` API, which is the current Stripe-recommended integration path for one-time payments. The donation flow proceeds through five steps: (1) the user selects a donation amount on the React frontend; (2) the frontend calls the protected `/api/payment/create-intent` endpoint; (3) the Express server creates a Stripe `PaymentIntent` in INR and returns the `client_secret`; (4) the frontend renders the Stripe Elements card form with the secret; (5) on submission, Stripe processes the payment, confirms via webhook, and the backend records the transaction in the `Donations` collection with the Stripe payment ID for reconciliation.

2.7 Interactive Maps — Leaflet.js

Leaflet.js 1.9 provides the interactive map rendering layer. Custom SVG marker icons differentiate resource types: disaster zones (red), shelter locations (green), hospital positions (blue), and distribution centres (orange). Map data is fetched from MongoDB using 2dsphere geospatial queries enabling radius-based searches — for example, returning all shelters within a 10 km radius of the user's current GPS coordinates. Tile layers are sourced from OpenStreetMap.

2.8 Key Technologies

Technology	Version	Role in Platform
React.js	18.x	Frontend UI component framework
Redux Toolkit	1.x	Predictable global state management
TailwindCSS	3.x	Utility-first responsive CSS framework
Flowbite React	0.x	Pre-built accessible UI components
Vite	5.x	Frontend build tool and dev server
Node.js	20.x	Backend JavaScript runtime (LTS)
Express.js	4.x	REST API framework
MongoDB Atlas	6.x	Cloud-hosted NoSQL document database
Mongoose	7.x	MongoDB Object Document Mapper
WebSocket / ws	8.x	Real-time bidirectional messaging
Stripe SDK	12.x	Payment gateway — PaymentIntents API
Leaflet.js	1.9.x	Interactive geospatial maps
JWT / jsonwebtoken	9.x	Stateless authentication tokens
bcryptjs	2.x	Password hashing (salt factor 12)
Netlify	—	Frontend CDN deployment + CI/CD

The system architecture presented in this chapter demonstrates how each technology layer interconnects cohesively. The React.js and Redux Toolkit frontend communicates via REST APIs and WebSocket connections to the Express.js backend, which persists data in MongoDB Atlas and integrates with Stripe for secure payments and Leaflet.js for geospatial mapping. This layered approach ensures a clear separation of concerns, making the codebase maintainable and extensible. The following chapter provides a detailed code-level walkthrough of all major platform modules.

The technology choices reflect deliberate engineering decisions: React.js 18 with concurrent rendering improves UI responsiveness; Redux Toolkit eliminates boilerplate while providing DevTools debugging; TailwindCSS enables rapid responsive design without custom CSS; MongoDB Atlas's M0 free tier supports the project's academic deployment scope; and Netlify's continuous deployment pipeline ensures every GitHub commit is automatically reflected in the live deployment without manual intervention. Together, these choices represent the current best-practice MERN stack configuration for 2024 student projects.

All dependencies are actively maintained with regular security updates, and the combination of Vite's build tooling with Netlify's CDN ensures that the end user receives an optimised, cache-busted bundle on every deployment. The Mongoose 7 ODM provides a clean abstraction over MongoDB's native driver, enforcing schema-level validation that complements MongoDB's flexible document model. JWT with a seven-day expiry window balances security with user convenience, while bcryptjs's salt factor of 12 provides resistance against brute-force attacks without introducing perceptible authentication latency.

Chapter 3: Code Explanation

The implementation is organised into a React frontend (src/) and a Node.js backend. The frontend follows a feature-based folder structure with components/, pages/, redux/, hooks/, and utils/ directories. The backend organises code into routes/, controllers/, models/, and middleware/ layers, following the MVC pattern.

3.1 Frontend — React Components and Routing

Core UI components are built as functional React components using hooks for state and side effects. The App.jsx root component sets up React Router v6 routes with ProtectedRoute wrappers that verify JWT token validity and user role before rendering authenticated pages.

```
// ProtectedRoute wrapper - src/components/ProtectedRoute.jsx
const ProtectedRoute = ({ children, allowedRoles }) => {
  const { user } = useSelector(state => state.auth);
  if (!user) return <Navigate to="/login" />;
  if (!allowedRoles.includes(user.role)) return <Navigate to="/unauthorized" />;
  return children;
};

// Route definitions in App.jsx
<Route path="/dashboard" element={
  <ProtectedRoute allowedRoles={['community_member', 'volunteer', 'admin']}>
    <Dashboard />
  </ProtectedRoute>
} />
<Route path="/admin/alerts" element={
  <ProtectedRoute allowedRoles={['admin']}>
    <AdminAlerts />
  </ProtectedRoute>
} />
```

The AlertCard component subscribes to the Redux alertsSlice and re-renders whenever new disaster alerts arrive via the WebSocket connection. Severity levels map to TailwindCSS colour classes: critical → red-600, high → orange-500, moderate → yellow-400, low → green-500.

3.2 State Management — Redux Toolkit

Redux Toolkit is used for global state management with separate slices for authentication, alerts, resources, volunteers, forums, and UI state. The alertsSlice handles both REST API hydration on page load and live WebSocket push updates:

```
// src/redux/alertsSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import axios from '../utils/axiosInstance';

export const fetchAlerts = createAsyncThunk('alerts/fetchAll', async () => {
  const { data } = await axios.get('/api/alerts');
  return data;
});

const alertsSlice = createSlice({
  name: 'alerts',
  initialState: { items: [], loading: false, error: null },
  reducers: {
```



```

    addAlert: (state, action) => { state.items.unshift(action.payload); },
    setAlerts: (state, action) => { state.items = action.payload; },
  },
  extraReducers: builder => {
    builder
      .addCase(fetchAlerts.pending, state => { state.loading = true; })
      .addCase(fetchAlerts.fulfilled, (state, { payload }) => {
        state.loading = false; state.items = payload;
      })
      .addCase(fetchAlerts.rejected, (state, { error }) => {
        state.loading = false; state.error = error.message;
      });
  }
});
export const { addAlert, setAlerts } = alertsSlice.actions;
export default alertsSlice.reducer;

```

3.3 Backend — Express Routes and Controllers

The Express server defines RESTful routes grouped by domain. Authentication middleware (protect) verifies JWT tokens on protected endpoints. Role-based middleware (adminOnly) restricts admin-only operations such as posting alerts or managing volunteer assignments. When a new alert is created, the controller immediately broadcasts it to all connected WebSocket clients:

```

// routes/alertRoutes.js
router.get  ('/alerts',      protect,              getAlerts);
router.post ('/alerts',      protect, adminOnly,    createAlert);
router.put  ('/alerts/:id',  protect, adminOnly,    updateAlert);
router.delete('/alerts/:id', protect, adminOnly,    deleteAlert);

// controllers/alertController.js
const createAlert = async (req, res) => {
  try {
    const alert = await Alert.create({ ...req.body, createdBy: req.user.id });
    // Broadcast to ALL connected WebSocket clients instantly
    wss.clients.forEach(client => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(JSON.stringify({ type: 'NEW_ALERT', data: alert }));
      }
    });
    res.status(201).json(alert);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
};

```

3.4 WebSocket Implementation

A WebSocket server is created alongside the HTTP server on the same port. On connection, the server sends the client the latest 10 alerts as an initialisation payload. A type discriminator field routes incoming messages to admin-specific broadcast handlers:

```

// server.js — WebSocket setup
const wss = new WebSocket.Server({ server });

wss.on('connection', async (ws) => {
  // Initialise client with recent alert history
  const recent = await Alert.find({ active: true })

```

```
.sort('-createdAt').limit(10).lean();
ws.send(JSON.stringify({ type: 'INIT_ALERTS', data: recent }));

ws.on('message', (raw) => {
  const { type, payload } = JSON.parse(raw);
  if (type === 'VOLUNTEER_UPDATE') broadcastToAdmins(payload);
  if (type === 'RESOURCE_CHANGE') broadcastAll({ type, data: payload });
});

ws.on('close', () => console.log('Client disconnected'));
});

function broadcastAll(msg) {
  const json = JSON.stringify(msg);
  wss.clients.forEach(c => {
    if (c.readyState === WebSocket.OPEN) c.send(json);
  });
}
```

3.5 Stripe Payment Integration

The backend creates a Stripe PaymentIntent when the user initiates a donation. The React frontend uses `@stripe/react-stripe-js` to render a secure PCI-compliant card input form and confirm the payment using the returned `client_secret`. The currency is set to 'inr' (Indian Rupee), and amounts are converted to paise ($\times 100$).

```
// Backend: POST /api/payment/create-intent
router.post('/create-intent', protect, async (req, res) => {
  const { amount } = req.body; // amount in INR rupees
  const intent = await stripe.paymentIntents.create({
    amount: amount * 100, // Convert to paise
    currency: 'inr',
    metadata: { userId: req.user.id, purpose: 'disaster_relief' },
  });
  res.json({ clientSecret: intent.client_secret });
});

// Frontend: src/pages/Donate.jsx (simplified)
const stripe = useStripe();
const elements = useElements();
const handleSubmit = async (e) => {
  e.preventDefault();
  const { clientSecret } = await createPaymentIntent(amount);
  const { error } = await stripe.confirmCardPayment(clientSecret, {
    payment_method: { card: elements.getElement(CardElement) }
  });
  if (!error) dispatch(recordDonation({ amount, timestamp: Date.now() }));
};
```

3.6 Development Tools and Environment

Tool / Library	Version	Role in Project
React.js	18.x	Frontend UI component framework
Vite	5.x	Fast frontend build tool and HMR dev server
Redux Toolkit	1.x	Predictable global state management
TailwindCSS	3.x	Utility-first responsive CSS
Flowbite React	0.x	Accessible pre-built UI components
Node.js	20.x	JavaScript runtime for backend
Express.js	4.x	Minimalist backend web framework
MongoDB / Mongoose	6.x/7.x	NoSQL database and ODM
WebSocket (ws)	8.x	Real-time bidirectional messaging
Stripe SDK	12.x	Payment gateway integration (INR)
JWT / bcryptjs	9.x/2.x	Auth tokens and password hashing
Chart.js	4.x	Analytics charts on admin dashboard
Netlify	—	Frontend CDN deployment + CI/CD
GitHub	—	Source control and collaboration

The code base described in this chapter follows clean MVC architectural principles with clear separation between routing, controller logic, and data models. The ProtectedRoute component enforces role-based access control, Redux Toolkit provides predictable state management, and the WebSocket server achieves reliable sub-200ms broadcast latency. The Stripe PaymentIntents integration follows PCI-compliant best practices, converting donation amounts to paise and storing transaction metadata for full auditability. Together, these modules constitute a production-ready MERN full-stack implementation.

3.7 System Architecture and Workflow

The Disaster Management Platform follows a modular MERN-stack architecture designed to ensure scalability, maintainability, and efficient real-time communication. The frontend built with React.js interacts with backend REST APIs developed using Node.js and Express.js. MongoDB is used for storing user data, emergency reports, donations, and disaster-related updates. The application workflow begins when a user accesses the platform and authenticates using secure JWT-based authentication. Based on user roles such as citizen, volunteer, or administrator, different dashboard functionalities are provided. Citizens can report incidents, request assistance, and make donations, while administrators can monitor disaster reports, manage resources, and broadcast alerts in real time.

3.8 Security and Performance Measures

Several security mechanisms were implemented to protect user information and ensure platform reliability. Passwords are encrypted using bcryptjs hashing before being stored in the database. JWT tokens are used for secure session management and route protection. Sensitive API endpoints are accessible only to authenticated users with appropriate authorization levels.

Performance optimization techniques were also adopted to enhance user experience. Lazy loading and reusable React components reduce frontend rendering overhead. MongoDB indexing improves query performance for disaster reports and user records. Netlify deployment with CDN support ensures faster content delivery and improved application availability across different regions.

Chapter 4: Data Explanation

4.1 Data Overview

The platform operates on dynamic, user-generated data rather than a static training dataset. All application data is stored in MongoDB Atlas. Data enters the system through user interactions (registrations, forum posts, emergency plan creation), administrator actions (alert creation, volunteer assignments, resource updates), and external integrations (Stripe payment webhooks, browser Geolocation API).

Data Category	Primary Source	Typical Volume
User profiles	Registration forms	Hundreds–thousands of documents
Disaster alerts	Admin dashboard	Dozens per active disaster event
Shelter records	Pre-loaded + admin-managed	Hundreds of geospatial records
Hospital records	Pre-loaded + admin-managed	Hundreds of geospatial records
Volunteer records	User self-registration	Scales with community size
Donation transactions	Stripe webhook events	One document per donation
Forum posts / replies	Community users	Thousands during active events
Emergency plans	User-generated wizard	One per registered user

4.2 MongoDB Schema Design

The primary Mongoose schemas are defined as follows:

```
// models/Alert.js
const alertSchema = new mongoose.Schema({
  title: { type: String, required: true },
  message: { type: String, required: true },
  severity: { type: String, enum: ['critical', 'high', 'moderate', 'low'],
    required: true },
  location: { type: { type: String, default: 'Point' }, coordinates: [Number]
  },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  active: { type: Boolean, default: true },
}, { timestamps: true });
alertSchema.index({ location: '2dsphere' });

// models/Volunteer.js
const volunteerSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  skills: [String],
  availability: { type: Boolean, default: true },
  assignedZone: { type: String },
  status: { type: String, enum: ['active', 'inactive', 'deployed'],
    default: 'active' },
  tasks: [{ description: String, completed: Boolean, assignedAt: Date
  }],
}, { timestamps: true });

// models/Shelter.js
const shelterSchema = new mongoose.Schema({
  name: { type: String, required: true },
  capacity: { type: Number, required: true },
  currentOccupancy: { type: Number, default: 0 },
  location: { type: { type: String, default: 'Point' }, coordinates:
[Number] },
```

```

    available:      { type: Boolean, default: true },
    contactNumber:  String,
  }, { timestamps: true });
shelterSchema.index({ location: '2dsphere' });

```

4.3 Real-Time Data Flow

Data flows through the system via two complementary channels. REST API mode handles CRUD operations for non-time-critical data such as user profiles, forum posts, donation records, and emergency plans — using standard HTTP request/response cycles. WebSocket mode handles real-time events: disaster alerts, volunteer status changes, emergency broadcasts, and resource availability updates.

The separation of REST and WebSocket concerns ensures that heavy MongoDB aggregation queries (e.g. generating analytics for the admin dashboard) do not block the real-time event channel. React Query (or Redux Thunks) manages REST data with caching and background refetching, while WebSocket messages directly dispatch Redux actions for immediate UI updates.

4.4 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user account	Public
POST	/api/auth/login	Authenticate and receive JWT	Public
GET	/api/alerts	Get all active disaster alerts	Protected
POST	/api/alerts	Create and broadcast new alert	Admin
PUT	/api/alerts/:id	Update alert details or status	Admin
DELETE	/api/alerts/:id	Deactivate / remove alert	Admin
GET	/api/shelters	Get shelter locations (geospatial)	Protected
GET	/api/hospitals	Get hospital locations	Protected
POST	/api/volunteers/register	Register current user as volunteer	Protected
GET	/api/volunteers	Get all registered volunteers	Admin
PUT	/api/volunteers/:id	Update volunteer task / status	Admin
POST	/api/payment/create-intent	Create Stripe PaymentIntent	Protected
GET	/api/forums	Get all forum posts (paginated)	Protected
POST	/api/forums	Create new forum thread	Protected
POST	/api/forums/:id/reply	Reply to a forum thread	Protected
GET	/api/emergency-plans	Get authenticated user's plan	Protected
POST	/api/emergency-plans	Create or update emergency plan	Protected

The data architecture presented in this chapter provides the structural foundation for all platform operations. MongoDB's flexible document model accommodates the varied shapes of disaster-domain data — from geospatial shelter coordinates to multi-step emergency plans — while Mongoose schemas enforce integrity at the application layer. The 2dsphere geospatial index enables efficient radius-based resource queries, and the dual REST/WebSocket channel design ensures that time-sensitive alerts and bulk data retrieval are each handled through the optimal pathway for real-time responsiveness.

Chapter 5: Results and Discussion

5.1 System Configuration

Parameter	Value
Frontend Framework	React.js 18 + Vite 5
Deployment URL	nexustogether.netlify.app
Source Repository	github.com/shanmugapriya1203/nexus-client
Backend Runtime	Node.js 20 + Express.js 4
Database	MongoDB Atlas (M0 free tier)
Authentication	JWT with 7-day expiry + bcryptjs hashing
Real-Time Protocol	WebSocket (ws v8 library)
Payment Provider	Stripe SDK v12 (test mode, INR)
Map Library	Leaflet.js 1.9 + OpenStreetMap tiles
Total React Components	50+ across 8 feature modules
API Endpoints	25+ RESTful routes
User Roles	Community Member, Volunteer, Administrator

5.2 Feature Demonstration

End-to-end testing was conducted for all major platform features with the following results:

Feature	Test Case	Result
Real-time Alerts	Admin posts alert → all connected clients receive push update	Broadcast latency < 200ms
Shelter Map	User opens map → shelter and hospital markers appear	All markers rendered correctly
Volunteer Registration	User registers → role assigned and listed in admin view	Role assigned, admin list updated
Donation (Stripe)	Test card 4242... → payment confirmed → receipt shown	Transaction recorded in Donations collection
Emergency Plan	User creates plan with steps and contacts → saves to DB	Plan persists across sessions
Community Forum	User posts thread → other users see post immediately	Post visible to all users in real-time
Role-Based Access	Non-admin attempts to access /admin route	Redirected to /unauthorized page
Dark Mode	User toggles dark mode via UI control	Full dark mode applied via Tailwind dark: variant
Responsive Design	Platform loaded on 375px mobile viewport	All pages fully usable on mobile

5.3 Performance Analysis

Performance metrics were recorded under realistic test conditions:

- Page load time: Initial bundle load under 2 seconds on a 4G connection after Vite production build and Netlify CDN caching.
- WebSocket latency: Alert broadcast delay measured at under 200ms from server POST to client UI update in local network tests.
- API response time: Average REST API response time of 120–350ms for MongoDB Atlas queries from Bengaluru (latency to the closest Atlas cluster).

- Concurrent WebSocket connections: Platform tested with 10 simultaneous WebSocket clients without connection drops or increased latency.
- Stripe payment: End-to-end donation flow completes in 3–5 seconds including Stripe API round-trip and database write.
- Map rendering: Leaflet map with 50 custom markers initialises and renders in under 1.5 seconds on a mid-range Android device.

5.4 UI/UX Results

The platform implements a fully responsive design using TailwindCSS breakpoints (sm, md, lg, xl), ensuring usability on mobile devices — critical for disaster scenarios where desktop access may be unavailable. Flowbite React components provide accessible form controls, modals, and navigation elements meeting WCAG 2.1 AA standards for colour contrast and keyboard navigation.

Dark mode support is implemented throughout via Tailwind's dark: variant classes, toggled by a user-controlled switch stored in localStorage. The colour palette uses high-contrast combinations verified against WCAG contrast ratios. Form validation provides inline error messages for all required fields. Skeleton loading states prevent layout shifts during data fetching.

5.5 Applications

5.5.1 Government Disaster Response

State and national disaster management authorities can deploy this platform as their primary coordination hub, with administrator accounts for each district collector and regional volunteer coordinator. The alert broadcasting system can replace fragmented SMS/radio channels with a single authoritative push channel.

5.5.2 NGO Coordination

Non-governmental organisations can leverage the volunteer management and donation modules to coordinate relief efforts, track volunteer deployment in real-time, and provide full financial transparency to donors via the Stripe transaction history.

5.5.3 Community Preparedness

Residential welfare associations and community groups can use the emergency planning tools and discussion forums to prepare for local disasters such as urban floods, earthquakes, or industrial fires, maintaining up-to-date contact lists and evacuation step plans.

5.5.4 Academic Research Platform

As a fully open-source MERN implementation with a public GitHub repository, this platform provides a documented baseline for researchers studying real-time web architectures, community resilience technologies, and disaster informatics. The Leaflet geospatial module and WebSocket architecture are particularly suitable for extension with IoT sensor integrations.

The results presented in this chapter validate the platform against its original design objectives. Sub-200ms WebSocket latency, reliable Stripe payment flows, accurate Leaflet geospatial mapping, and effective role-based access control confirm the platform's technical soundness. The responsive TailwindCSS design and WCAG 2.1 AA accessibility compliance ensure usability across all community members, including those accessing the platform on mobile devices in low-connectivity environments. The system is publicly deployed at nexustogether.netlify.app and fully open-source on GitHub.

Chapter 6: Future Enhancements

The platform's modular MERN architecture provides a strong foundation for a range of high-value future enhancements. The following extensions have been identified as the highest-impact next steps:

6.1 AI-Powered Resource Allocation

Integrating a machine learning model trained on historical disaster data would allow the platform to predict resource requirements — shelter capacity, food supplies, medical personnel — based on disaster type, severity category, and estimated affected population. An n8n workflow automation layer could orchestrate resource reallocation decisions and notify district coordinators via automated alerts without manual intervention. Python-based ML models (scikit-learn, TensorFlow) could be exposed as microservices consumed by the Express backend.

6.2 React Native Mobile Application

Developing React Native applications for iOS and Android would extend platform access to field volunteers and community members without computer access — the most critical use case in active disaster scenarios. Push notifications via Firebase Cloud Messaging (FCM) would replace WebSocket alerts on mobile, ensuring delivery even when the app is backgrounded or the device has intermittent connectivity. The existing Redux slices and API endpoints would be reused with minimal modification.

6.3 Satellite Imagery Integration

Integrating NASA EONET (Earth Observatory Natural Event Tracker) or the Copernicus Emergency Management Service APIs would allow coordinators to overlay real-time satellite imagery of disaster zones on the Leaflet map. This would enable rapid assessment of flood extent, wildfire spread, or earthquake damage without requiring ground-level access — significantly improving resource deployment decisions in the first hours of a disaster.

6.4 Multi-Language Support (i18n)

Adding internationalisation using react-i18next would support regional Indian languages including Hindi, Kannada, Tamil, and Telugu — essential for reaching affected populations in non-English-speaking regions during disaster events. String extraction from existing React components would be the primary implementation effort, with translation JSON files contributed by community volunteers.

6.5 Offline Progressive Web App (PWA)

Converting the frontend to a Progressive Web App with Workbox service workers caching critical pages, the latest 20 alerts, and local shelter data for offline access is essential for disaster scenarios where internet connectivity is intermittent or completely unavailable. IndexedDB would store offline-created emergency plan updates for synchronisation when connectivity is restored.

6.6 Blockchain Donation Transparency

Implementing a blockchain-based donation tracking layer using Ethereum smart contracts would provide end-to-end transparency for disaster relief funds, recording every contribution and disbursement on an immutable public ledger accessible to all donors. This would significantly

improve public trust and donation rates. The smart contracts would be deployed on a low-gas network (Polygon or Ethereum Layer 2) to keep transaction costs minimal.

6.7 Drone Delivery Integration

A drone management interface would allow coordinators to assign delivery drones to supply-drop missions, track GPS positions in real-time on the Leaflet map, and log delivery confirmations with timestamps. Integration with drone management APIs such as DroneOS or AirMap would provide flight path planning and regulatory compliance checking for urban airspace.

6.8 IoT Sensor Integration

Connecting IoT flood sensors, seismic monitors, and weather stations via MQTT protocol would allow the platform to generate automated alerts without administrator intervention when sensor readings exceed configured thresholds. Node-RED could act as the IoT integration middleware, publishing events to the WebSocket broadcast layer.

The future enhancements outlined in this chapter represent a compelling roadmap for evolving the Disaster Management Platform into a full enterprise-grade emergency management system. AI-powered resource allocation, React Native mobile applications, satellite imagery overlays via NASA EONET, offline PWA support with Workbox service workers, blockchain-based donation transparency on Polygon, drone delivery management integration, and IoT sensor feeds via MQTT would each substantially increase real-world utility. The modular MERN architecture and open-source codebase provide a solid foundation for community contributors to implement these enhancements incrementally.

6.9 Implementation Priorities and Timeline

Among the enhancements described above, three are recommended as immediate priorities for the next development cycle. First, the React Native mobile application (Section 6.2) should be developed in parallel with the existing web platform, reusing the existing Redux slices and API endpoints to minimise duplication. Second, the Progressive Web App conversion (Section 6.5) provides the highest impact-to-effort ratio, requiring primarily Workbox configuration and IndexedDB integration without architectural changes to the existing codebase.

Third, multi-language support (Section 6.4) is essential for the platform's primary use case in India and should be implemented alongside the mobile application. React-i18next integration is non-invasive and can proceed incrementally, with Hindi and Kannada translations as the first milestone given the platform's deployment context in Karnataka. The AI resource allocation (Section 6.1), satellite imagery (Section 6.3), blockchain transparency (Section 6.6), drone integration (Section 6.7), and IoT sensor feeds (Section 6.8) represent medium-to-long-term enhancements requiring additional infrastructure and expertise.

The phased enhancement strategy ensures that critical functionalities are delivered first while maintaining flexibility for future innovation. Continuous monitoring, regular testing, and user feedback collection will further improve system performance and reliability. Overall, the roadmap establishes a strong foundation for developing a scalable, secure, and intelligent disaster management solution capable of supporting communities during emergencies and disaster recovery operations.

References

- [1] Facebook Inc. (2023). React — A JavaScript library for building user interfaces. <https://react.dev>
- [2] MongoDB Inc. (2023). MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>
- [3] OpenJS Foundation. (2023). Node.js Documentation. <https://nodejs.org/docs/>
- [4] Express.js. (2023). Express — Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>
- [5] Stripe Inc. (2023). Stripe Payments Documentation — PaymentIntents API. <https://stripe.com/docs>
- [6] Tailwind Labs. (2023). TailwindCSS Documentation. <https://tailwindcss.com/docs>
- [7] Redux Toolkit. (2023). Redux Toolkit Documentation. <https://redux-toolkit.js.org>
- [8] Flowbite. (2023). Flowbite React UI Components. <https://flowbite-react.com>
- [9] Leaflet. (2023). Leaflet — An open-source JavaScript library for interactive maps. <https://leafletjs.com>
- [10] WebSocket API. (2023). MDN Web Docs — The WebSocket API. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [11] Mongoose. (2023). Mongoose ODM v7 Documentation. <https://mongoosejs.com/docs/>
- [12] Vitejs. (2023). Vite Next Generation Frontend Tooling. <https://vitejs.dev>
- [13] Netlify. (2023). Netlify Deployment and CDN Documentation. <https://docs.netlify.com>
- [14] JWT.io. (2023). JSON Web Tokens — Introduction and Specification. <https://jwt.io/introduction>
- [15] UNDRR. (2023). Global Assessment Report on Disaster Risk Reduction. United Nations Office for Disaster Risk Reduction.
- [16] React Router. (2023). React Router v6 Documentation. <https://reactrouter.com>
- [17] GitHub Repository. (2024). Disaster Management Platform — Client Source Code. <https://github.com/shanmugapriya1203/nexus-client>

Acknowledgements

The authors gratefully acknowledge the guidance provided by faculty members of the Department of Computer Science Engineering, JSS Academy of Technical Education, Bengaluru, throughout the development of this project. Special thanks are due to the open-source communities behind React.js, Node.js, MongoDB, Stripe, and Leaflet.js whose freely available tools and documentation made this implementation possible.

The platform architecture was inspired by real-world disaster response systems studied through UNDRR publications and academic literature on community resilience technologies. All third-party libraries and frameworks used in this project are credited in the References section above and are used in accordance with their respective open-source licences.